

User Guide: LinkedIn Job Finder

Documentation of Python Scripts, Search Parameters, AI-Assisted Workflow, and File Export

Author: Javier Medina Moreno (2026) | License: GNU AGPLv3

General Summary: LinkedIn Job Finder is an automated data acquisition and curation suite composed of four modular Python scripts designed to streamline professional job hunting. It manages the entire lifecycle: discovering matching vacancies across public guest endpoints (`Finder.py`), extracting comprehensive role specifications (`Extractor.py`), partitioning dossiers into token-friendly batch chunks (`Splitter.py`), and assembling high-priority opportunities while pruning rejected listings (`Filter.py`). All network interactions query LinkedIn guest pages directly without requiring accounts, login credentials, or private API keys, while atomic disk writes and immediate write-flushing guarantee zero loss of previously collected data.

TABLE OF CONTENTS

1. What is LinkedIn Job Finder?	1
2. Pipeline Architecture and Operational Workflow	2
3. Internal Code Structure	4
4. Generated Files and Data Flow	5
5. Step-by-Step User Guide	6
6. Authorship and License of Use	6

1. What is LinkedIn Job Finder?

It is a data acquisition framework developed to eliminate the manual friction of scouting professional vacancies. Rather than spending hours clicking through paginated web cards, opening dozens of ephemeral browser tabs, and copying text fragments manually, this pipeline executes systematic vacancy queries directly against LinkedIn's public guest infrastructure.

By bypassing heavy headless browser environments (such as Selenium or Playwright) in favor of optimized HTTP sessions, the tool provides high throughput, deterministic execution, and complete isolation from your personal LinkedIn profile. Output dossiers are automatically sanitized, structured, and segmented for direct evaluation with Large Language Models (LLMs) to perform automated candidate-to-job matching and dossier curation at scale.

2. Pipeline Architecture and Operational Workflow

2.1. Discovering Job Postings (Finder.py)

- **Boolean Keyword Matching:** Evaluates complex logical expressions (combining degrees, technical tools, and titles with AND/OR/NOT operators) paired with target locations.
- **Geodesic Radial Search (`DISTANCE`):** Translates geographic radius into LinkedIn's native distance filter, evaluated strictly in statute miles (valid increments: `0`, `5`, `10`, `25`, `50`, `100`).
- **Configurable Pagination Ceiling (`MAX_PAGES`):** Controls search depth by specifying an integer page limit (10 jobs per page) or setting `None` to traverse up to LinkedIn's public guest ceiling of 1,000 listings.
- **Native URL Parameter Mapping:** Automatically maps posting timeframe (`TIME_FILTER`), seniority (`EXPERIENCE_LEVEL`), workplace arrangement (`WORKPLACE_TYPE`), and contract type (`JOB_TYPE`).
- **Atomic Disk Persistence:** Writes every newly discovered vacancy immediately to CSV with explicit buffer-flushing, guaranteeing zero data loss if execution is paused or interrupted.

Note on Keywords: Keyword selection establishes the absolute boundary of your discovery pool. Any vacancy not matched by your Boolean search query will never enter the pipeline for extraction and analysis. However, overly broad or excessive keywords cause output bloat, prolong extraction runtimes, and risk prematurely reaching LinkedIn's 1,000-listing public guest limit with low-signal postings. Calibrate terms to ensure broad coverage of target domains without degrading search precision.

2.2. Extracting Full Descriptions (Extractor.py)

- **Dual-Mode Input Handling:** `TARGET_URLS` accepts full LinkedIn URLs, bare 8+ digit numeric Job IDs (e.g., `4469886861`), or mixed collections, standardizing them via `normalize_job_url()` .
- **Inline Formatting Protection:** The DOM parser converts `<br>` tags to line breaks and appends newlines strictly to block containers (`<p>` , `<li>` , headings, `<tr>`), preventing inline tags from fragmenting sentences.
- **Per-Job Immediate Flush Persistence:** Appends each extracted role to disk with an explicit `f.flush()` call after every job, ensuring complete fault tolerance if interrupted via `Ctrl+C` .
- **Resumable State Auditing:** Scans destination dossiers prior to issuing network requests, skipping completed IDs to eliminate duplicate traffic.
- **Autonomous Chaining:** Triggers automatically upon `Finder.py` completion when enabled, exporting to user-configured paths (`DETAILS_OUTPUT_FILENAME`).

2.3. Batch Splitting & AI Curation (Splitter.py & Filter.py)

- **Header-Free Batch Chunking:** Partitions massive dossiers into uniform chunks (default: 50 jobs) named `linkedin_job_details_batch_*.txt` that omit metadata headers and begin directly with `JOB #1: ...` to conserve model context.
- **Dual-List State Management:** `Filter.py` processes both `INTERESTING_JOB_IDS` (inclusion) and `EXCLUDED_JOB_IDS` (blacklisting), emitting console warnings if any ID appears in both sets simultaneously.
- **Self-Modifying Source Auto-Pruning:** When list collisions occur, `Filter.py` programmatically modifies its own source code on disk, stripping conflicting IDs from `INTERESTING_JOB_IDS` while preserving the exclusion list.
- **Dossier Pruning & Gapless Renumbering:** Evaluates existing `interesting_jobs.txt` files, deletes blacklisted positions, and rewrites remaining entries with contiguous, gapless sequential numbering (`JOB #1` , `JOB #2` , ...).

Note on AI Curation: Because screening depends on how effectively job listings are evaluated, prompt instructions should be crafted carefully according to your particular background, constraints, and goals.

AI acts as the primary filter in this pipeline, so ensure the chosen AI model provides sufficient context length and reasoning depth to process job batches consistently.

Recommended Model: Gemini Flash with Extended Thinking.

Example of Screening Prompt:

Analyze my attached CV against the provided job listings to select only the roles that represent a genuine, realistic fit for my profile and that I should apply to.

My Profile & Objectives:

- Experience Level: Recent graduate with 0 years of professional experience.
- Short-Term Objective: Obtain hands-on, bottom-up experience and exposure in the labor market.
- Long-Term Target: Roles leading into business strategic analysis, commercial strategy, FP&A, pricing, or strategic consulting.

Evaluation Criteria:

1. Real Acceptance Odds: The company must be genuinely open to entry-level profiles.
2. Career Development: Roles offering structured learning, exposure to business decisions, and analytical rigor over manual administrative tasks.
3. Future Earnings Potential: Starting points that build high-value commercial or quantitative skills.

Rules:

- You must evaluate all provided jobs.
- You must evaluate each job individually.
- You should include all relevant job IDs in the output. There is no limit to how many jobs you can include.

Strict Filtering Rules (Eliminate any role that fails these constraints):

- Strict Title Filter: EXCLUDE any role with Senior, Lead, Manager, Director, Head, Principal, Staff, or VP in the title (even if the body text mentions fewer years; avoid title-inflation traps).
- Hard Experience Cap: EXCLUDE any role that requires years of mandatory professional experience. ONLY keep roles specifying 0-1 years, 0-2 years, formal Graduate Programs, Traineeships, Apprenticeships, Internships, or where experience is explicitly labeled as optional ("a plus", "valorable", "desirable").
- Education Cap: EXCLUDE any role that explicitly mandates a Master's degree or PhD. (Master's degrees are acceptable only if explicitly listed as an optional plus or if the program itself funds/provides the Master's).
- Payload Integrity: ONLY select job IDs that actually appear in the text payload provided in this prompt. Do not import or retain IDs from previous conversations or external lists.

Output the selected IDs in this exact format:

```
INTERESTING_JOB_IDS = ""  
[Job ID]  
[Job ID]  
[Job ID]  
""
```

3. Internal Code Structure

The pipeline decouples high-speed discovery from deep specification parsing and stateful curation across four specialized Python modules:

Script & Function	Purpose and Role in the Workflow
<code>Finder.py: User Input Fields</code>	Configures query keywords, location, geodesic distance radius (statute miles), pagination limit (<code>MAX_PAGES</code>), date filters, experience level, auto-run handoff, and output path.
<code>Finder.py: scrape_linkedin_jobs()</code>	Paginates public guest listings up to <code>MAX_PAGES</code> / 1,000 jobs with radial geodesic filtering, flushes new listings to CSV atomically, and manages handoff.
<code>Extractor.py: normalize_job_url()</code>	Extracts canonical 8+ digit numeric IDs via regex and standardizes arbitrary links or standalone Job IDs into uniform guest endpoints.
<code>Extractor.py: get_existing_job_ids()</code>	Reads destination text files to construct an in-memory set of completed Job IDs for safe, resumable extraction.
<code>Extractor.py: parse_linkedin_job_url()</code>	Fetches endpoints, normalizes block line breaks (preserving inline formatting tags), strips UI button artifacts, and manages HTTP 429 throttling back-off.
<code>Extractor.py: parse_multiple_job_urls()</code>	Orchestrates bulk extraction sequentially, applies stochastic politeness delays (2.5 to 5.5s), writes and flushes (<code>f.flush()</code>) each job immediately, and cleanly traps interrupts.
<code>Splitter.py: split_job_details()</code>	Partitions large master dossiers into standalone batch files of N jobs (default: 50), omitting top metadata headers and starting directly with <code>JOB #1: ...</code> .
<code>Filter.py: parse_source_job_blocks()</code>	Indexes master text dossiers into regex-bounded blocks accessible directly by their numeric Job ID.
<code>Filter.py: parse_job_blocks_ordered()</code>	Preserves ordered sequence of (<code>job_id</code> , <code>block</code>) tuples to maintain stable sorting during dossier manipulation.
<code>Filter.py: prune_conflicts_from_source()</code>	Self-modifying source cleaner: programmatically removes any Job ID present in <code>EXCLUDED_JOB_IDS</code> from <code>INTERESTING_JOB_IDS</code> directly inside <code>Filter.py</code> on disk.
<code>Filter.py: prune_excluded_jobs()</code>	Audits existing <code>interesting_jobs.txt</code> dossiers, strips blacklisted jobs, and rewrites remaining entries with gapless sequential numbering (<code>JOB #1</code> , <code>JOB #2</code> , ...).
<code>Filter.py: filter_interesting_jobs()</code>	Extracts selected positions, prevents duplicate additions, appends new records, and coordinates collision detection and self-modifying source pruning.

4. Generated Files and Data Flow

Data moves across four structured persistence layers designed to maintain modularity and complete state safety throughout execution:

4.1. Search Results Spreadsheet (`Finder.py` → `linkedin_jobs.csv`)

Description: Standardized tabular catalog consolidating vacancy cards discovered during pagination.

Key elements: Clean schema (`job_id`, `title`, `company`, `location`, `date_posted`, `url`) with automatic in-memory deduplication and immediate file-flushing.

4.2. Master Details Dossier (`Extractor.py` → `linkedin_job_details.txt`)

Description: Full role specifications, responsibilities, and requirements compiled across targeted vacancies.

Key elements: 80-character boundary dividers (`=====`), clean paragraph structure, stripped UI artifacts, and immediate per-job `f.flush()` writes for crash resilience.

4.3. Partitioned Batch Segments (`Splitter.py` → `linkedin_job_details_batch_*.txt`)

Description: Standalone text chunks partitioned from master dossiers, sized specifically for LLM prompt context injection.

Key elements: Configurable batch sizing (default: 50 jobs per chunk), range-indexed filenames (e.g., `linkedin_job_details_batch_1_jobs_1-50.txt`), and direct start at `JOB #1: ...`.

4.4. Final Curated List (`Filter.py` → `interesting_jobs.txt`)

Description: Curated application dossier containing exclusively approved positions, automatically synchronized with blacklists.

Key elements: Gapless consecutive renumbering (`JOB #1`, `JOB #2`, ...), automatic deletion of excluded IDs, and duplicate prevention.

4.5. Built-in Politeness and Safety Controls

Description: Multi-tiered operational protections that keep scraping sustainable, polite, and indistinguishable from regular browsing.

Key elements: Randomized request intervals (2.5 to 5.5s), automatic 35-second cooldown upon HTTP 429 (Too Many Requests), and clean interrupt handling via `Ctrl+C`.

5. Step-by-Step User Guide

- 1. Set Up Your Search (Finder.py):** Open `Finder.py` and configure parameters under `USER_INPUT_FIELDS`: `KEYWORD` (e.g., `'(Economics OR Business) AND ("Power BI" OR Python)'`), `LOCATION` (e.g., `'Barcelona, Catalonia, Spain'`), `DISTANCE` = `'25'` (statute miles: `0`, `5`, `10`, `25`, `50`, `100`), `MAX_PAGES` (integer ceiling or `None` for 1,000 jobs), `TIME_FILTER` (`'r604800'`), and `EXPERIENCE_LEVEL` (`'1,2'`). Set `AUTO_RUN_EXTRACTOR` = `True` and verify `DETAILS_OUTPUT_FILENAME`.
- 2. Run Discovery Search:** Open your terminal and run `python Finder.py`. Postings append immediately to `linkedin_jobs.csv`. If enabled, Extractor launches automatically upon completion.
- 3. Download Full Descriptions (Extractor.py):** To extract manually or target specific listings, assign standalone Job IDs or full links to `TARGET_URLS`, or keep `INPUT_CSV` = `'linkedin_jobs.csv'`, then run `python Extractor.py`. Existing IDs are automatically skipped, and each job is flushed to disk immediately.
- 4. Split Dossier for LLM Ingestion (Splitter.py):** Verify `BATCH_SIZE` = `50` in `Splitter.py` and run `python Splitter.py`. This creates segmented chunk files (e.g., `linkedin_job_details_batch_1_jobs_1-50.txt`) that start directly with `JOB #1: ...`.
- 5. Screen Jobs with an AI LLM Model:** Upload your CV alongside as many batch files as can fit comfortably within the context window of your AI model of choice. Write a prompt indicating your background constraints, current situation, and target career goals (or use the example screening prompt provided in Section 2). Ensure you specify the exact assignment syntax so the returned Job IDs can be pasted directly into `Filter.py`.
- 6. Assemble, Prune, and Renumber Application Dossier (Filter.py):** Paste the `INTERESTING_JOB_IDS` block returned by the AI into `Filter.py` (you can add the job IDs you have already applied to or discarded in `EXCLUDED_JOB_IDS`) and run `python Filter.py`. The script flags list collisions, self-prunes conflicting IDs from its own source code on disk, strips blacklisted vacancies from `interesting_jobs.txt`, and writes remaining opportunities with clean, gapless sequential numbering (`JOB #1`, `JOB #2`, ...).

6. Authorship and License of Use

Author: Javier Medina Moreno (2026)

License: GNU AGPLv3 (GNU Affero General Public License v3.0). Free to use, modify, and redistribute under the terms of the GNU AGPLv3. All derivative works and modifications must retain original author attribution and copyright notices.